

Instituto Tecnológico de Costa Rica
Escuela Ingeniería en Computadores
Curso: CE1101 Introducción a la
programación

Documentación técnica

Crazy Snack Rush

Estudiantes:

Ronald Mora Rojas
Fariathna Venegas Venegas

Profesor:

Leonardo Araya Martínez

Semestre I 2026

Índice

1. Introducción	4
1.1. Propósito del documento	4
1.2. Descripción del proyecto.....	4
1.3. Objetivos y Alcance.....	4
1.4. Definiciones, Acrónimos y Abreviaturas	5
2. Descripción General del Sistema.....	5
2.1. Funcionalidades principales.....	5
2.2. Restricciones del sistema, Supuestos y Dependencias.....	7
3. Requisitos del Sistema	9
3.1. Requisitos Funcionales	9
3.2. Requisitos No Funcionales	10
4. Diseño de la Solución:.....	10
4.1. Visión General de la Arquitectura	10
4.2. Descripción de Componentes.....	12
4.3. Stack Tecnológico	13
4.4. Diagrama de Clases y Modelo de Datos	14
4.5. Decisiones de Diseño y Justificación.....	16
5. Implementación y Validación de Calidad	17
5.1. Estructura del repositorio	17
5.2. Entorno de Desarrollo, Guía de instalación y Ejecución	17
5.3. Estrategias y Casos de Prueba	18
5.4. Resultados de pruebas y problemas encontrados	19
Resultados de pruebas y problemas encontrados	22
6. Declaración de uso de Inteligencia Artificial y herramientas	25
6.1. Herramientas de IA utilizadas	25
6.2. Declaración de Responsabilidad y Autoría.....	26
7. Conclusión	27

8. Bibliografía.....	28
----------------------	----

1. Introducción

1.1. Propósito del documento

El presente documento tiene como propósito describir la arquitectura, diseño, funcionamiento e implementación del juego Crazy Snack Rush, destinado principalmente a la lectura y entendimiento del profesor del curso.

1.2. Descripción del proyecto

Crazy Snack Rush, al cual, llamamos OvercookTED es un juego multijugador de escritorio, que busca crear el entorno de una cocina y simular el juego Overcooked. Los jugadores deberán utilizar la cocina y estaciones que se encuentran en ella para lograr crear la receta solicitante por los clientes, obteniendo durante el juego puntajes que se les asignarán.

Para el desarrollo del juego se utilizó Python Pygame para la interfaz gráfica y desarrollo completo de un juego 2D. Además, tuvo como paradigma de programación principal la Programación Orientada a Objetos.

1.3. Objetivos y Alcance

1.3.1. Objetivo General

Desarrollar un juego multijugador local en Python, utilizando herramientas que permitan simular el entorno de una cocina, haciendo que el usuario logre interactuar con elementos de su alrededor.

1.3.1. Objetivos Específicos

1. Diseñar el juego utilizando la librería Pygame para lograr una interfaz vistosa y coherente al juego.

2. Desarrollar un sistema que calcule puntajes y penalice errores.

3. Diseñar sprites con una herramienta externa para la creación del mapa según la temática de la cocina.

4. Implementar un sistema de movimiento mediante el teclado, permitiendo la interacción de los jugadores con elementos del mapa.

1.4. Definiciones, Acrónimos y Abreviaturas

- **Frontend:** El frontend se refiere a las tecnologías del lado del cliente, es decir, aquellas que se ejecutan en el navegador del usuario y con las que este interactúa directamente. Incluye principalmente HTML, CSS y JavaScript, y se enfoca en la experiencia del usuario, la interfaz y la usabilidad. [5]
- **Backend:** El backend corresponde al lado del servidor y se encarga de procesar la información, interactuar con bases de datos, gestionar sesiones de usuarios y servir el contenido al frontend. Utiliza lenguajes como PHP, Python, Java, entre otros. [5]
- **POO:** La POO organiza el programa en objetos en lugar de módulos o subrutinas, donde un objeto se define como una entidad del mundo real que cuenta con atributos y comportamientos. En la POO se definen clases, que funcionan como plantillas para crear objetos. [6]
- **Sprites:** Los sprites son imágenes o animaciones bidimensionales superpuestas en una escena. Son los elementos dinámicos de un juego 2D, que se mueven independientemente del fondo. <https://remarkablecoder.com/sprites-in-games/>
-

2. Descripción General del Sistema

2.1. Funcionalidades principales

- **Pantalla inicial:** Permite al usuario iniciar el juego
- **Selección de nivel:** El usuario elige entre Nivel 1, Nivel 2 o Nivel 3, cada uno con distinta dificultad. Cada nivel carga su propio mapas y recetas

- **Configuración de chefs:** Se crean dos objetos chef que son capaces de moverse con distintas teclas.
- **Mapa y tiles:** se cargan los tiles para crear los tres mapas y se usan para representar estaciones.
- **Sistema de estaciones:** Existen estaciones de despensa y de procesamientos: cocina, freidor y tabla de picar, cada una de estas tienen una función.
- **Procesamiento de ingredientes:** Las estaciones de procesamiento, tienen una lista de ingredientes aceptados y un tiempo determinado para que los ingredientes sean procesados.
- **Recoger y dejar ingredientes:** Los chefs pueden agarrar ingredientes de las estaciones de despensa y dejar sobre mesas vacías.
- **Sistema de platos:** Los chefs pueden agarrar un plato e ir agregando ingredientes a este.
- **Entrega de platos:** Los chefs puede entregar platos, la estación de entrega procesa los ingredientes recibidos y los compara con la lista de recetas pendientes, asigna un puntaje o penaliza.
- **Órdenes de recetas:** El sistema genera recetas con nombre, puntaje y tiempo límite. Estas se muestran en pantalla para tenerlas como referencia.
- **Temporizador de nivel:** Cada nivel tiene un tiempo un tiempo límite, cuando se acaba el tiempo también se acaba la partida.
- **Movimiento de chefs:** Los jugadores o ches se mueven por el mapa utilizando el teclado, cada chef utiliza teclas distintas

- **Gestión de eventos:** Cuando se toca una tecla de teclado se detecta un evento. Cada acción desencadena lógica específica.
- **Colisiones y cercanía:** Se colocan ciertos tiles como obstáculos que los chefs no pueden sobrepasar, además cuando se presiona la tecla de acción se detecta el tipo de estación más cercano.
- **Sistema de mesas:** Las mesas permiten colocar ingredientes o platos, y recogerlos después. Se usan tiles específicos para cada estado del ingrediente.
- **Control de estados:** Mediante un atributo de la clase ingredientes se maneja estados de los ingredientes como “crudos” o “preparados”.
- **Finalización de partida:** Cuando el tiempo llega a cero, el bucle principal termina y se cierra el nivel.

2.2. Restricciones del sistema, Supuestos y Dependencias

Restricciones:

- El juego debe ser desarrollado exclusivamente en el lenguaje de programación Python.
- Debe tener una interfaz gráfica que permita al usuario interactuar con el programa.
- El sistema se ejecuta localmente con los recursos de la computadora.

Supuestos:

- El usuario cuenta con Python instalado en su computadora.
- El usuario tiene instaladas las dependencias necesarias, especialmente Pygame.
- El usuario cuenta con un entorno de desarrollo. Preferiblemente Visual Studio Code.
- El usuario ha clonado el proyecto desde GitHub.
- El usuario cuenta con un teclado con flechas.

Dependencias:

- Librerías: Pygame y Sys
- Python 3.x
- Folders con los tiles
- Imágenes del repositorio en GitHub

3. Requisitos del Sistema

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	El sistema permite que los dos chefs se muevan con sus respectivas teclas	Alta	Cada chef se logra mover sin interrupción alguna
RF-002	Se debe mostrar una receta cada cierto tiempo en la pantalla, y esta debe durar un tiempo limite	Alta	Se muestran las recetas en pantalla y cada una de estas tienen un tiempo de finalización.
RF-003	El chef debe poder agarrar ingredientes de las estaciones de despensa	Alta	El chef logra agarrar ingredientes
RF-004	El chef debe poder dejar ingredientes sobre mesas vacía	Alta	Si el chef tiene un ingrediente en la mano, logra poder dejarlo en una mesa vacía
RF-005	El chef debe poder dejar ingredientes en estaciones de procesamiento y estos se deben procesar con la duración establecida.	Alta	El ingrediente se procesa en estaciones de procesamiento
RF-006	El chef debe añadir ingredientes en un plato	Alta	El chef añadió los ingredientes en el plato en mano.
RF-007	El chef debe poder entregar el plato en la estación de entrega para obtener puntos.	Alta	Si el plato coincide con una orden activa, se suman los puntos al chef correspondiente si no, se penaliza.

RF-008	El sistema debe tener tres niveles con mapas y recetas distintos.	Media	Cada nivel carga su propio world_data, estaciones y recetas temáticas.
RF-009	Durante el juego se debe representar visualmente cada ingrediente	Media	Los ingredientes se modifican al interactuar.
RF-009	Finalización de la partida	Media	Cuando se acaba el tiempo de partida, esta termina

3.2. Requisitos No Funcionales

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Usabilidad	Los controles deben ser intuitivos y deben ser mostrados antes de iniciar el juego	Más del 90% logren utilizar el juego sin ningún tipo de ayuda
RNF-002	Disponibilidad	Funciona para cualquier computador con Python y dependencias instaladas	El juego funciona sin errores

4. Diseño de la Solución:

4.1. Visión General de la Arquitectura

OvercookTec sigue una arquitectura monolítica orientada a objetos, organizada en módulos Python separados por responsabilidad. La capa de presentación está manejada por Pygame mediante el uso de tiles, rectángulos, imágenes, etc. La lógica del juego reside en las clases de Clases.py (Chef, Ingrediente, Plato, Receta, Cocina, Estacion); y los datos del mundo se gestionan en mundo.py (Mundo). El archivo main.py actúa como controlador principal que integra todos los módulos y maneja el ciclo de juego.

Se eligió arquitectura monolítica porque el juego es de alcance limitado, no requiere comunicación en red y la integración estrecha entre lógica y presentación es más simple de mantener en este contexto, también, se basa en el conocimiento técnico de los integrantes. Cualquier otro tipo solo añadiría dificultad.

Diagrama de Arquitectura.



4.2. Descripción de Componentes

Componente	Responsabilidad	Tecnología	Dependencias
Interfaz Gráfica (GUI)	Permite la interacción del usuario con el juego, mostrar los personajes en pantalla, mediante tiles y rectángulos permiten representar estaciones	Pygame, LibreSprite	Backend
Backend o lógica del Juego	Controlar el movimiento de los chefs, colisiones existentes, los pedidos. Administrar ingredientes, estados de preparación, cocción y validación de recetas.	Python (Programación Orientada a Objetos)	Pygame Datos del juego
Gestión del Mundo y Estaciones	Controlar mapas, estaciones de cocina, interacción con objetos y distribución de elementos en el escenario.	Python Pygame	Pygame Datos del juego Lógica del Juego
Datos del juego	Contener recursos visuales como las imágenes de fondo o tiles que representan mapas	Carpetas: "tileset", "imágenes", "" y archivos PNG	Pygame

4.3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Lenguaje	Python	3.14.3	Lógica del juego, manejo de archivos, manejo de diccionarios, manejo de funciones y POO.
Interfaz Gráfica o motor gráfico (GUI)	Pygame	2.x	Mostrar mapas, utilizando los tiles, detección de eventos de teclado, control del tiempo y actualización de la pantalla.
Diseño de mapas	LibreSprite y Uso de archivos PNG	1.1	Permite el dibujado y creación de tiles como archivos PNG a gusto de los integrantes.
Entorno de desarrollo	Visual Studio Code	1.117.0	Edición, ejecución y pruebas del código.
Otras Librerías	OS	Incluidas en Python	OS: Gestión de rutas de los archivos (función cargar imagen)
Trabajo en equipo	Github	-	Control de cambios, trabajo colaborativo y respaldo del código fuente.

4.4. Diagrama de Clases y Modelo de Datos

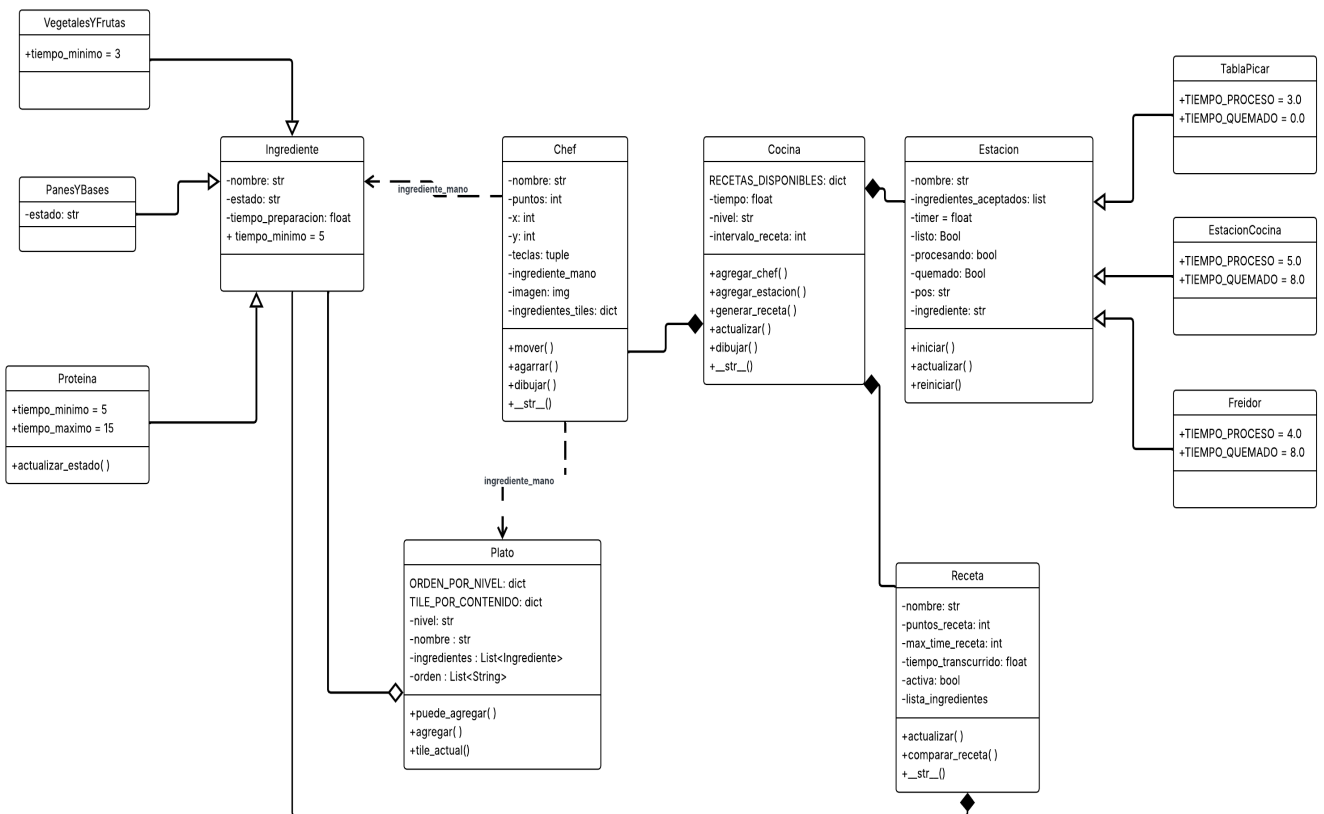
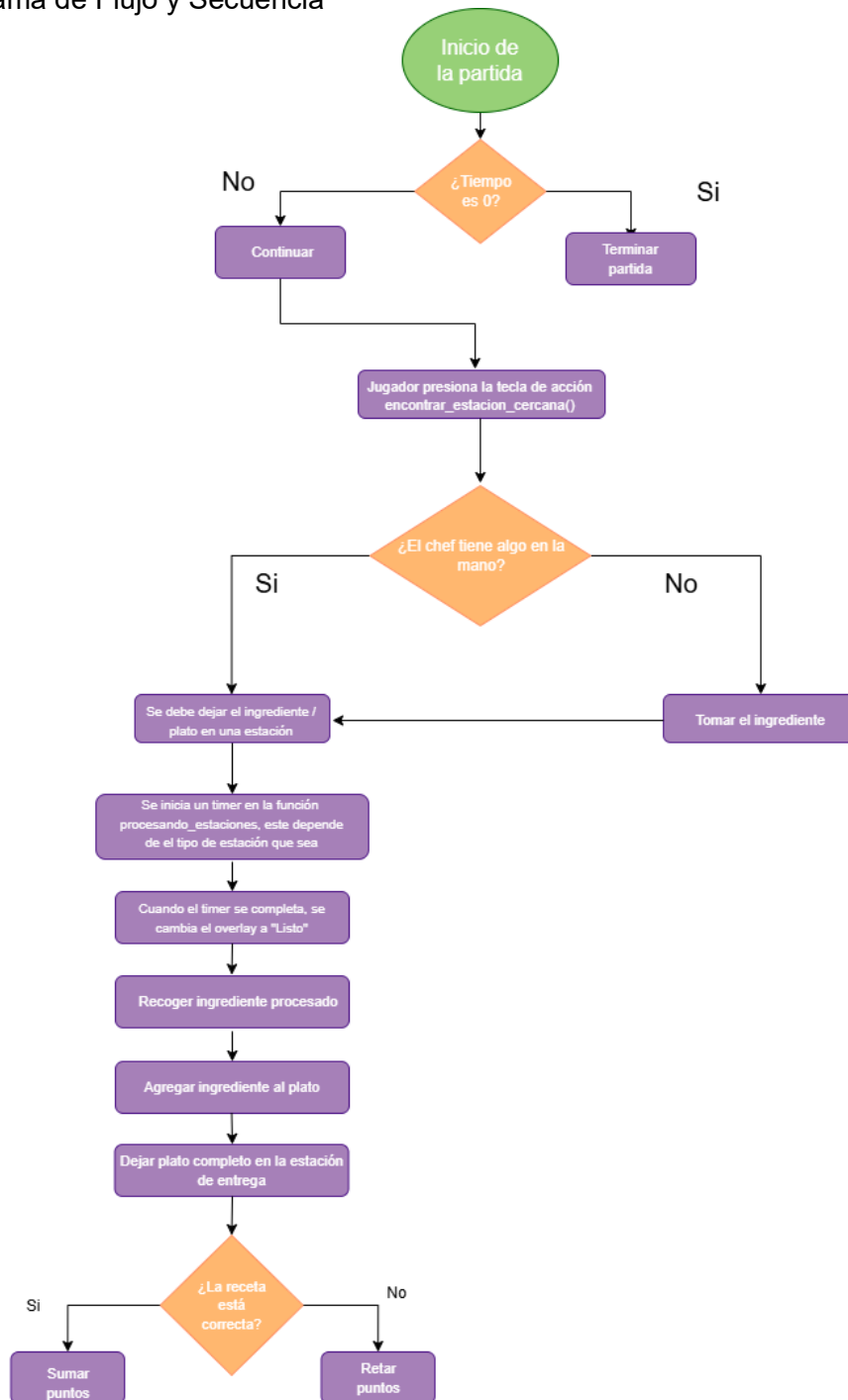


Diagrama de Flujo y Secuencia



4.5. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Desarrollo	Visual Studio Code	Visual Studio Code	Se seleccionó Visual Studio Code debido a sus herramientas de autocompletado, accesibilidad, facilidad y compatibilidad con Python y Pygame. Además, era un requisito del proyecto.
Arquitectura del sistema	Programación Orientada a Objetos	Programación Orientada a Objetos	Principalmente se emplearon por ser uno de los requisitos a evaluar. También era la mejor opción para este tipo de videojuegos. El uso de clases facilitó la reutilización de código mediante herencia, encapsulamiento y polimorfismo.
Interfaz gráfica	• Tkinter • Pygame	Pygame	Se eligió Pygame porque ofrece herramientas específicamente para el desarrollo de videojuegos 2D, incluyendo manejo de sprites, colisiones y eventos de teclado.
Diseño gráfico	• Inteligencia artificial • Sprites descargados de internet • Hacerlos propios	Hacerlo propios con Libresprite	Para lograr un resultado como el que se planteó al inicio, se tenía que tener acceso directo al diseño de los escenarios, ingredientes, etc... Por ello se optó por crear diseños propios en Libresprite.

5. Implementación y Validación de Calidad

A continuación, se describe como se estructura el repositorio donde se encuentra el código, el proceso de instalación de las dependencias del proyecto y las estrategias de pruebas que se utilizó durante el desarrollo de OvercookTEC

5.1. Estructura del repositorio

```
/Proyecto-OvercookTEC → Carpeta principal
├─ mundo.py → Código fuente principal del juego
├─ Clases.py → Archivo de persistencia de puntajes
├─ Constantes.txt → Archivo de persistencia de puntajes
├─ Mundo.txt → Archivo de persistencia de puntajes
├─ /sushi_tiles → Contiene los tiles referentes al nivel 1
└─
├─ /tilest → Contiene los tiles referentes al nivel 1
└─
├─ /cr_tiles → Contiene los tiles referentes al nivel 1
```

5.2. Entorno de Desarrollo, Guía de instalación y Ejecución

- **Entorno:**
 - Sistema Operativo: Windows 10 o superior
 - Lenguaje: Python 3.x
 - Editor: Visual Studio Code 1.117.0
 - Interfaz Gráfica: Pygame
 - Librerías: Pygame, Sys, OS
- Guía de instalación y ejecución:
 1. **Clonar el repositorio**
git clone <https://github.com/fariathna28/Proyecto-OvercookTec.git>
 2. **Instalar Python**
Descargar desde <https://www.python.org/downloads/>
Elige tu sistema operativo.
Marca la opción 'Add Python to PATH'
 3. **Instalar Pygame**
Abrir el símbolo del Sistema (CMD)
coloque el comando: pip install Pygame

5.3. Estrategias y Casos de Prueba

Los siguientes cuadros describe la estrategia general de pruebas que se utilizó en el proyecto y cuáles fueron los casos de prueba más importantes que se implementaron. Cabe recalcar que no se utilizó ningún software de prueba o validaciones, todas las pruebas se realizaron con la ejecución manual del programa en Visual Studio Code.

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación de funcionalidades individuales. Principalmente pruebas de cada clase y cada método.	Python y VS Code	>= 80% del código
Pruebas de integración	Validación de las interacciones entre los chefs y todos los elementos de la cocina.	Python y VS Code	100% del programa
Pruebas de funcionamiento	Verificación de las funcionalidades (entrega, cocina, puntaje, movimiento, etc...)	Python y VS Code	Todo el programa, principalmente el flujo del archivo main.py
Pruebas de interfaz	Se verifica que todas las pantallas funcionen correctamente, es decir, que se dibujen y funcionen todos los tiles, imágenes, etc... de forma correcta.	Python, VS code, Pygame	100% de las pantallas

5.4. Resultados de pruebas y problemas encontrados

Casos de prueba:

ID	Descripción	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-00 1	Pantalla de inicio	1. Ejecutar programa	Aparece una pantalla con el logo del juego y un botón de "Comenzar".	Funciona	☒ Pasó
CP-00 2	Pantalla de menús	1. Ejecutar el programa 2. Presionar el botón "comenzar"	Se muestra una pantalla con los 3 niveles para escoger.	Funciona	☒ Pasó
CP-00 3	Pantalla del nivel	1.Presionar el botón según el nivel que se quiera jugar.	Se abre la pantalla con la interfaz gráfica del nivel según cada temática.	Funciona	☒ Pasó
CP-00 4	Movimiento de chefs	1.Comenzar el juego 2.Presionar las teclas que controlan a cada chef (A, S, D, W) y (Flecha arriba, flecha abajo, flecha derecha y flecha izquierda)	Ambos chefs se mueven a todas direcciones de forma correcta al presionar las letras asignadas para cada chef.	Funciona	☒ Pasó
CP-00 5	Obtener ingredientes y plato de las estaciones	1.Comenzar juego 2.Acercar el chef a una estación 3.Presionar la tecla de acción (E o Shift-Izq)	El chef debe tomar en su mano lo que esté en la estación, ya sea un ingrediente o un plato. Se debe mostrar de una dimensión más pequeña en la mano del chef.	Funciona	☒ Pasó
CP-00 6	Dejar ingrediente o plato en una mesa	1.Tomar un ingrediente o plato 2.Acercarse a una mesa y dejar el	El ingrediente o plato debe aparecer sobre la mesa con la que el chef esté colisionando.	Funciona	☒ Pasó

		ingrediente en alguna mesa libre con la letra de acción (E o Shift-Izq).			
CP-007	Cocina	1.Comenzar el juego 2.Tomar un ingrediente de la categoría “proteína” 3.Dejar la proteína en la cocina con la letra de acción.	La proteína elegida debe de cocinarse al pasar 5 segundos. Después de 10 segundos, esta se quema.	Funciona	☒ Pasó
CP-008	Freidora	1.Comenzar el juego 2.Tomar un ingrediente que se tenga que freír 3. Dejar el ingrediente en la freidora	El ingrediente elegido debe de cocinarse al pasar 4 segundos. Después de 8 segundos, esta se quema.	Funciona	☒ Pasó
CP-009	Tabla de picar	1.Comenzar el juego 2.Tomar un ingrediente que se tenga que picar. 3. Colocar el ingrediente en la tabla de picar	El ingrediente a picar debe de picarse al pasar 3 segundos.	Funciona	☒ Pasó
CP-010	Generar receta	1.Comenzar la partida	Al iniciar la partida debe de generarse una lista de recetas aleatorias.	Funciona	☒ Pasó
CP-011	Armar plato	1.Comenzar la partida 2.Preparar los ingredientes según corresponda (cortarlos, freírlos o cocinarlos)	La receta se irá armando en el plato según los ingredientes que se vayan tomando.	Funciona	☒ Pasó

		3. Tomar un plato de la estación de platos 4. Con el plato en mano, ir tomando los ingredientes según el orden de las recetas reales.			
CP-012	Entrega de los pedidos	1. Realizar una de las recetas del pedido. 2. Dejar el plato en la estación de entregas	El pedido es entregado con éxito, desaparece de las manos del chef y se añade un puntaje según si la entrega es correcta o no.	Funciona	☒ Pasó
CP-013	Asignación de puntajes	1. Dejar el plato en la estación de entregas.	Si la receta es correcta, se le suma un puntaje al chef, este puntaje varía según la cantidad de ingredientes que esta lleve. Por otro lado, si se entrega una receta incorrecta se debe de restar al puntaje que se lleva, de igual forma varía según la receta.	Funciona	☒ Pasó
CP-014	Terminar la partida	1. El tiempo llega a 0	Se muestra una puntuación final con los puntajes	Funciona	☒ Pasó

Resultados de pruebas y problemas encontrados

Durante todo el desarrollo del sistema se estuvieron haciendo muchas pruebas constantemente. Cuando se presentaba un error o en general algún aspecto del sistema no funcionaba, se buscaron formas para resolver estos inconvenientes, para finalmente tener un videojuego completamente funcional.

- Resultados:

- El 100% de los casos unitarios funcionaron.
- Los dos chefs se mueven a todas direcciones según las teclas asignadas.
- Los escenarios para cada nivel se dibujan correctamente.
- Se asigna un tiempo de partida acorde a cada nivel.
- Se dibujan los chefs
- Se generan recetas de forma aleatoria, con un tiempo de vencimiento
- Las interacciones entre el chef, los ingredientes y las estaciones (tiles) funcionan correctamente.
- Los ingredientes se procesan de forma correcta (tabla de picar, cocina, freidora) según el tiempo asignado.
- En el caso de la cocina y la freidora, los ingredientes se queman después de cierto tiempo transcurrido.
- El estado de los ingredientes cambia después de pasar por un proceso.
- El sistema de puntuaciones suma o resta puntos según la entrega realizada.
- La partida se finaliza al terminar el tiempo.

- Problemas encontrados:

Problemas Solucionados: Durante el desarrollo del proyecto se presentaron numerosos problemas que se lograron solucionar, a continuación, se mencionan los más relevantes.

1. Detección incorrecta de estaciones de trabajo:

Error: Los chefs no interactuaban correctamente con las estaciones de ingredientes.

Causa: Algunas posiciones y dimensiones de los objetos no coincidían correctamente dentro del mapa. (Las colisiones se realizan respecto los rectángulos de la estación y el chef)

Solución: Se utilizó coordenadas centrales de los objetos y se verificó la distancia entre el chef y las estaciones para determinar la interacción correcta.

2. Errores al colocar y retirar ingredientes de las mesas:

Error: Los ingredientes desaparecían o no podían volver a recogerse después de ser puestos sobre una mesa.

Causa: No se actualizaba el tile.

Solución: Se realizaron significativos cambios a la función *actualizar_estaciones()*. Principalmente en la lógica del algoritmo.

3. Validación incorrecta de recetas completas:

Error: Al entregar una receta correcta se restaban puntos.

Causa: El orden en que los ingredientes eran agregados al plato podía variar, provocando diferencias durante la comparación.

Solución: Se restringió la receta a tener un orden específico.

4. Tiempos del juego inconsistentes:

Error: Algunos ingredientes terminaban su preparación antes o después del tiempo indicado en el código.

Causa: Los tiempos de preparación de algunos ingredientes no se actualizaban de forma completamente precisa, lo que ocasionaba que los ingredientes terminaran de procesarse antes o después de lo esperado.

Solución: Se utilizó el valor delta generado por `clock.tick(60)` para controlar los tiempos de preparación

Problemas no solucionados: A continuación, se presenta una lista de los problemas encontrados que no se pudieron solucionar.

1. Interacciones muy dependientes de la posición del chef:

Error: El jugador debe colocarse en una posición muy específica para interactuar con una estación.

Causa: El sistema de interacción utiliza colisiones entre rectángulos, por lo que pequeñas diferencias en la posición del personaje pueden afectar la detección.

¿Por qué no se solucionó?: Para corregir el error tendríamos que cambiar todo el sistema de colisiones, por lo que no era viable.

2. No se guarda el proceso:

Error: El juego no conserva puntuaciones ni progreso después de cerrarse.

Causa: No se implementó un sistema de base de datos para almacenar el proceso.

¿Por qué no se solucionó?: No era un requisito, pero si sería una mejora a futuro.

6. Declaración de uso de Inteligencia Artificial y herramientas

6.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Período de Uso	Propósito
ChatGPT	Asistente de desarrollo.	Mayo/Junio 2026	Generación de ideas, sugerencias para optimizar el código y solución de bugs.
Copilot	Asistente de desarrollo.	Mayo 2026	Generación de imágenes.
Claude	Asistente de programación	Mayo/Junio 2026	Generación de ideas para código, solución de bugs, sugerencias para optimizar el código y mejorar la documentación técnica.
Gemini	Asistente de desarrollo.	Mayo/Junio 2026	Generación de imágenes.

1.1. Descripción del Uso de Inteligencia Artificial

1. ChatGPT: Esta herramienta se utilizó principalmente para entender mejor el funcionamiento de pygame, ya que ninguno de los integrantes tenía conocimientos previos de la librería. Se pedían explicaciones de ciertos comandos y sugerencias de cuales eran las opciones mas óptimas para la implementación de la librería. Adicionalmente se utilizó para la resolución de bugs no visibles fácilmente.

2. Copilot: Esta herramienta se utilizó solamente para generar la imagen de la pantalla de bienvenida del juego.

3. Claude: Claude se utilizó para resolver las dudas más técnicas del desarrollo del código, además de sugerencias de como realizar ciertas funciones más avanzadas de las cuáles no teníamos conocimientos. Fue de mucha ayuda para encontrar soluciones a los bugs más complejos y sugerencias para simplificar el código. Finalmente, nos brindó sugerencias para mejorar la documentación técnica.

4. Gemini: Esta herramienta se utilizó únicamente en la creación de los íconos para los niveles.

6.2. Declaración de Responsabilidad y Autoría

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto PokeTEC, declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

Firmas:

Ronald M. R.



7. Conclusión

Para concluir, podemos afirmar que el proyecto se ha concretado de forma exitosa, pudiendo abarcar todos los objetivos planteados desde el inicio. Se desarrolló un juego completamente funcional, inspirado en el videojuego “Overcooked”, donde se diseñaron 3 diferentes niveles de juego con todas las funcionalidades correspondientes.

Los dos integrantes del equipo coincidimos en decir que el mayor reto del proyecto fue aprender a usar la librería Pygame, ya que fue algo completamente nuevo para ambos. De igual forma, se decidió diseñar (dibujar desde 0 en un programa) toda la gráfica de los niveles, menos los chefs.

Finalmente, es importante mencionar que durante este proceso se adquirieron muchos aprendizajes técnicos, como, por ejemplo: Uso de clases, implementación de funciones, desarrollo en pygame, etc... También hubo mucho aprendizaje a la hora de trabajar en equipo. Se desarrollaron habilidades de comunicación, resolución de problemas en equipo y apoyo.

8. Bibliografía

- [1] BUcSWQxY0MI, “Curso completo de Pygame,” YouTube. [Video en línea]. Disponible en: <https://www.youtube.com/watch?v=BUcSWQxY0MI>. [Accedido: 16-jun-2026].
- [2] Z_7COcLeYFY, “Cómo hacer mapas en LibreSprite,” YouTube. [Video en línea]. Disponible en: https://www.youtube.com/watch?v=Z_7COcLeYFY. [Accedido: 16-jun-2026].
- [3] Zg10c0zycVw, “Creación de mapas en LibreSprite,” YouTube. [Video en línea]. Disponible en: <https://www.youtube.com/watch?v=Zg10c0zycVw>. [Accedido: 16-jun-2026].
- [4] BwWHXN2rZHQ, “Rectángulos en Pygame,” YouTube. [Video en línea]. Disponible en: <https://www.youtube.com/watch?v=BwWHXN2rZHQ>. [Accedido: 16-jun-2026].
- [5] Z0yLerU0g-Q, “Diagrama de clases POO,” YouTube. [Video en línea]. Disponible en: <https://www.youtube.com/watch?v=Z0yLerU0g-Q>. [Accedido: 16-jun-2026].
- [6] SI7O81GMG2A, “Diagramas de clases en programación orientada a objetos,” YouTube. [Video en línea]. Disponible en: <https://www.youtube.com/watch?v=SI7O81GMG2A>. [Accedido: 16-jun-2026].
- [7] “Pygame Tutorial,” *GeeksforGeeks*. [En línea]. Disponible en: <https://www.geeksforgeeks.org/python/pygame-tutorial/>. [Accedido: 16-jun-2026].
- [8] “Pygame Tutorial,” *TutorialsPoint*. [En línea]. Disponible en: <https://www.tutorialspoint.com/pygame/index.htm>. [Accedido: 16-jun-2026].
- [9] “Python | Moving an Object in Pygame,” *GeeksforGeeks*. [En línea]. Disponible en: <https://www.geeksforgeeks.org/python/python-moving-an-object-in-pygame/>. [Accedido: 16-jun-2026].
- [10] “Collision Detection in Pygame,” *GeeksforGeeks*. [En línea]. Disponible en: <https://www.geeksforgeeks.org/python/collision-detection-in-pygame/>. [Accedido: 16-jun-2026].